

CSC443: Database System Technology

Project Report

MRK–DB

Submitted by

Sanchaai Mathiyarasan
Anbuselvan Ragunathan
Yathusan Koneswararajah

University of Toronto, Fall 2025

How to Use

We have created a basic test suite that runs some unit tests for the database, as well as some experiments. In the top-level directory, running the `make` command should generate five executables: One for our unit tests (`tests`) as well as 4 for our experiments (`experiments_bin_search_btree`, `experiments_get`, `experiments_scan`, `experiments_put`). Running the unit tests executable will output the unit test results to the terminal, while running any of the latter executables will create a folder called `experiment_results` containing various CSVs corresponding to each experiment's results.

One can also create & open their own instance of MRK-DB by doing the following in a C++ file:

```
DB db; db.Open("databaseName");
```

If the database does not exist, then a new one (with default parameters) will be created. Otherwise (if a database with the given name already exists), that database will be loaded with the parameters it was originally created with.

It is also worth mentioning that when creating a new database, the `Open()` function also accepts optional parameters to customize the database configuration. These parameters are as follows:

- `bool useBTreeSearchValue;`
 - Whether or not to use the B-Tree to search for a page in an SST
- `uint64_t bitsPerEntryValue;`
 - The number of bits in the bloom filter that should be allocated for every key
- `uint64_t cacheSizeValue;`
 - The maximum number of pages in the buffer pool
- `uint64_t memtableThresholdValue;`
 - The maximum number of keys that need to be stored in the memtable to trigger a flush to the disk

Implementation

We have implemented all required materials that were outlined in the document. The implementation of the project itself is centred around a basic LSM-tree based key-value store, where each level of the tree is structured as an SST as a static B-tree. We now will delve into the details behind our design elements, project status, and testing.

Class Descriptions

We now will explain the purposes of each class, and how they serve their purposes throughout the project.

DB (include/mrkdb.hpp, src/mrkdb.cpp)

We have a DB class located in mrkdb.hpp and implemented in mrkdb.cpp. This class is the API that is exposed to the user and holds the buffer pool, LSM tree, and name of the database. When closing the database, the class holds a file named “meta.sst”, which contains the number of levels in the LSM tree, followed by a series of uint64’s that are set to either 0 or 1 depending on whether or not a level was occupied by an SST or not. The user is able to call: Open, Put, Get, Scan, Delete and Close on this database, all of which work in conjunction with the LSM tree.

Memtable (include/memtable.hpp, src/memtable.cpp)

Following this, we have a Memtable class located in memtable.hpp and implemented in memtable.cpp. The purpose of this class gives us the ability to structure our memtable as a balanced binary search tree (in this project, we decided to use the AVL tree as our balanced binary search tree). This class also acts as the buffer for the LSMTree (which will be discussed later), thereby supporting buffered insertions. Overall, this class supports the insertions, deletions, and retrievals of key-value pairs and automatically flushes to disk when the number of entries reaches the memtable’s threshold. The memtable makes use of a node class also found within the same files to allow the representation of nodes in the aforementioned AVL tree.

LSMTree (include/lsmtree.hpp, src/lsmtree.cpp)

We also have an LSMTree class located in lsmtree.hpp and implemented in lsmtree.cpp. This class gives us our LSMTree implementation that allows us to structure this KV-store as a write-optimised structure. It holds a memtable, a list of SST objects, a buffer pool, and the scale factor of the LSM tree (which will always be 2 since we are dealing with a basic LSM-tree).

The LSMTree works in conjunction with the DB class to take care of the KV-store API functions, thereby abstracting away the logic of handling the implementation logic of the DB API functionalities from the user.

***BloomFilter* (include/bloomfilter.hpp, src/bloomfilter.cpp)**

Moreover, we also made a BloomFilter class located in bloomfilter.hpp and implemented in bloomfilter.cpp. This class gives us our implementation of a bloom filter, allowing us to efficiently test if an element actually exists within a level of the LSM tree with a high degree of probability while using very little memory. To help with aspects of hashing, we used an implementation found online of the XXHash64 algorithm (citation provided below). This ensures that hash collisions are mitigated. With this class, we are efficiently able to add and check for keys to aid in our KV-store API operations.

***BTree* (include/btree.hpp, src/btree.cpp)**

Furthermore, we made a BTree class located in btree.hpp and implemented in btree.cpp. This class gives us our implementation of the SST files located at each level of the LSM tree as a static B-tree, allowing for efficient storage and retrieval of data from disk. We employ the usage of a B-tree node that we have defined as a BTreeNode (located in memtable.hpp), so that we can represent nodes in the aforementioned B-tree. A BTreeNode consists of the following information: the number of delimiting keys, a vector of the delimiting keys themselves, and a vector of children to the BTreeNode. It is worth noting that a BTreeNode always takes up one page of space.

***SSTViewer* (include/sstviewer.hpp, src/sstviewer.cpp)**

We also make use of an SSTViewer class located in sstviewer.hpp and implemented in sstviewer.cpp. This class gives us a way to efficiently look through the different SST. Naturally, this works in conjunction with our LSM tree class so that we are able to look for the key value pairs we wish to seek in an efficient manner. This class employs the help of the bloom filter to check if a certain key exists within its SST (synonymously, the level), thereby ensuring that there are no false negative lookups.

***SSTWriter* (include/sstwriter.hpp, src/sstwriter.cpp)**

In addition, we also defined an SSTWriter class located in sstwriter.hpp and implemented in sstwriter.cpp. This class allows us to give an abstraction of writing to SSTs to the LSM tree class. The SSTWriter class also uses the BloomFilter class so that it can write the keys into a BloomFilter such that it would aid with the lookups of the aforementioned keys when required.

BufferPool (include/bufferpool.hpp, src/bufferpool.cpp)

Finally, we also implemented a BufferPool class located in bufferpool.hpp and implemented in bufferpool.cpp. This class allows us to improve the overall performance of our database by introducing the notion of caching data pages. Essentially, we can cache these pages into main memory so that they can be accessed much faster when compared to accessing it for the disk. The database will first check to see if the data is already in memory (via the buffer pool). If so, the database avoids the slow disk I/O, thereby increasing the speeds of queries overall. Note that within the buffer pool, we use an open addressing hash table (meaning that it is represented as a vector), wherein it employs Robin Hood hashing for linear probing. For every cached page, we maintain: the page ID, the dirty bit, reference bit, the probe sequence length (PSL) and the actual cached page itself. This is all maintained and stored in a class called HPage.

Constants (include/constants.hpp, src/constants.cpp)

Note that we also have defined a file of some helper functions and constants and placed it in a header file called constants.hpp and implemented in a file titled constants.cpp. The functions defined were binary search and a calculation function for determining the number of items in a page. We deemed as a group that these functions would be required throughout the entire project, hence their placement into a general file such as this. The constants in the file are also served throughout the project and include values such as: $\ln 2$ (used for calculating the optimal number of hash functions), page size, a TOMBSTONE value used for deletions (which we have defined as the `UINT64_MAX` value), the size of a `uint64` in both bytes and bits, the number of `uint64`'s that can be fit into a page, and other macro functions that are used for otherwise rote calculations that are required throughout the project.

Globals (include/globals.hpp, src/globals.cpp)

There also is a file with some global variables defined in globals.hpp and implemented in globals.cpp. These global variables are used in the `Open()` function found in `mrkdb.hpp` and implemented in `mrkdb.cpp` and are used as the default values as required for the configuration of the database. These global variables include: `useBTreeSearch` (used for determining if we used the indexed B-Tree search or the non-indexed binary search), `bitsPerEntry`, `cacheSize` (used for the buffer pool), and a `memtableThreshold`.

SST Structure

Before continuing, we will discuss the structure of our SSTs. It follows as such:

- The metadata page which contains the number of keys, the minimum key, the maximum key
- The sorted leaf nodes which contains the actual contiguously stored key value pairs themselves, sorted in the order of keys
- The bloom filter, which follows after the sorted leaf nodes and stored statically in the SST. The bloom filter is created when the SST is created, and is stored as an array of uint64's. We get and set bits to by computing a hash value which can be used to compute the page offset, uint64 index and the bit within the uint64 that needs to be used
- The internal B-Tree nodes, where the pages for layer 1 are followed by the pages for layer 2, etc.

Design Elements

The KV-store's get, put and scan APIs, which are found in the files mrkdb.hpp/cpp with function name Get, Scan, and Put, calls the Get, Scan, and Put functions in lsmtree.hpp/cpp. Note that for everything that concerns the notion of reading or writing (which is all of our data structures in this project), we employ the usage of a buffer pool which acts as our gateway for all reads and writes that happen throughout our database. This buffer pool is implemented as a hash table and for collision resolution, we use Robin Hood hashing such that we can obtain linear probing. Moreover, when we reach capacity in the buffer pool, we use the clock eviction policy so that we evict the required pages such that there is enough space to support for new insertions into the buffer pool.

Gets and Scans

For both Get and Scan, we first query the in-memory memtable to see if the value(s) that we are attempting to find are within the memtable layer. The memtable itself as mentioned before is structured as a balanced binary search tree. More specifically, we have the memtable structured as an AVL tree. If the value is not found in the memtable (or not all of the values are found within the memtable), then we go ahead and search the SSTs that are in storage. For this, we make use of either binary search or B-Tree search (depending on the database configuration) since we have our SSTs sorted and structured as static B-trees. This allows us to have an efficient searching methodology to find (or determine that we have not found) a value within an SST. Specifically for Gets and Scans, we ensure that we make use of the bloom filter so that it can quickly tell us whether or not a key is within a level (or synonymously, that level's SST) or not, Note that the chances of false negatives with the Get functionality are non-existent due to the existence of our bloom filter (false positives are still

possible, however, due to the nature of bloom filters). After the binary search or the B-Tree search (in order to find the correct page), to find the position of the key within the page, another binary search takes place.

To implement Scan specifically, we followed the LSM tree scan algorithm that was outlined in class. For each level, we issued a buffer via the usage of the SSTViewer class. Then we search for the start of the key range within each level and continue until we reach the end of the range of the keys. While we are doing this loop, we ensure that we are outputting the youngest version of the entry with the next smallest key. And also if we traverse the last entry in a given buffer, we read the next page from the run. We ensure that we return the most recent version of each entry in the range across the entire LSM tree by scanning the SSTs in the order from newest to oldest (thus naturally getting more recent versions of an entry since the candidate newer versions of an entry would exist on newer SSTs whereas their older counterparts would exist in the older SSTs).

Note that we also have integrated the buffer pool with our Get and Scan functionalities. Essentially, we implemented caching via the buffer pool so that we can cache the data pages in the main memory, which results in a faster access time. The process is as follows: if the database requires a page, it will first check our bufferpool to see if it exists in memory, and if so, the database is able to skip over the slow disk I/O, thus speeding up our queries. Pages are also marked as dirty within the buffer pool when modified and are then written back to disk later so that the updated value can be persisted

Puts, Updates, & Deletes

For Put, we make use of buffered insertions via the memtable. If we see that the threshold of the memtable has been reached, we then flush this memtable down to the disk, wherein we write the memtable data into an SST. create a temporary SST used to help with compaction purposes, and merge the temporary SST and the original SST together. During the compaction process, the keys from the two input SSTs are sorted via multi-way merge sort. The bloom filter and internal B-Tree nodes are created right after this and stored in the SST. To optimize our performance, we create our bloom filter in memory (as stated is allowed in a Piazza post) and we skip over bloom filter and internal node creation if we are creating a temporary file which will eventually be merged with the next level. Following the merge, the temporary SST is discarded and replaced by the newly merged SST. Note that further, the Put operation also acts as our support for updates and deletes. Specifically for deletes, we have a function aptly titled Delete() in mrkdb.hpp and implemented in mrkdb.cpp. This function essentially just takes in the key of the key-value pair that is required to be deleted and calls the Put() method in lsmtree.cpp with the value being the TOMBSTONE value. For updates, we just call Put() as usual in mrkdb.cpp with the key that we want to update along with the new value

that we want the key to refer to, which is then passed along to `lsmtree.cpp`'s `Put()` method. For both updates and deletes, we ensure that we return the most recent insertion of the key's value by scanning the SST's from newest to oldest.

Opening and Closing

We also have an open and close API within our database as well, located in `mrkdb.hpp` and implemented in `mrkdb.cpp`. For the open functionality, the function is aptly named `Open()` within the files, and for close it is titled `Close()`. When opening the database, we first ensure the existence of the database that is attempted to be opened; if it already exists, then we populate the needed configuration variables from the "meta.sst" file and the user is able to continue on with using their database as they wish. If we deal with a new database, then we populate the database with either the default configuration variables or the user defined variables. Both paths will construct a buffer pool and an LSM tree that will be ready to be used throughout the database. It is worth mentioning that all data and data structures that are in re-opened databases are persisted when they are closed, meaning that there is no data (and internal states of data structures) that are lost when the user decides to terminate a session with a database.

When closing the database, we write the configuration variables of the database to the "meta.sst" file along with the status of the LSM tree. Specifically, we note down the number of levels the LSM tree had along with whether or not each level was occupied by an SST. This information is then useful when reopening the database, as mentioned before, the user can then continue on with their session with their database without worry of data loss.

Unit Tests

Our unit tests cover the full functionality of the database, thereby ensuring the correctness and robustness of our LSM-tree based key-value store. We have developed an extensive suite of unit tests that cover all the major functionalities of the DB class layer. This includes: operations on the memtable, interactions with the SST's, multi-level spanning queries, and edge case behaviours. The tests both validate functional correctness and ensure the persistence logic as the entries move from memory to the disk.

Testing Opening, Closing and Persistence

We have tests that cover the opening and closing of the database. The first test, `testDBOpen()` simply checks to see if the database exists after opening it. There is also

testDBCcloseFully(), which ensures that the database was closed properly after it was opened. testDBReopenGet() tests the persistence of data after a database has been closed and reopened. Similarly, testDBPersistentDeletes() tests the persistence of deletes after a database has been closed and reopened. testDBPersistentUpdates() also tests the persistence of data, but specifically for when a user decides to update a value, close the database, and reopen it.

Testing Gets and Scans

Throughout our test suite, we have a myriad of tests that tackle both point queries (gets) and range queries (scans). Further, we ensured that these tests covered both gets and scans within the memtable and the SSTs (i.e. testing retrievals from memory and from storage). These tests include:

For memtable gets:

- testDBGetMemtable()
- testDBGetNonExistentMemtable()

For SST gets:

- testDBGetSST()
- testDBGetSSTDeeper()
- testDBGetNonExistentSST()
- testDBGetNonExistentSSTDeeper()
- testDBReopenGet()

For memtable scans:

- testDBScanMemtable()
- testDBScanMemtableEmpty()

For SST scans:

- testDBScanSST()
- testDBScanSSTDeeper()
- testDBScanSSTEmpty()
- testDBScanAcrossSSTs()
- testDBScanGetMultipleKeysNotThere()

Testing Deletes

The test suite also considers the case of deleting entries from the database. Not only does it consider the case of deleting just within an open database, it also considers the case

where we close a database and reopen it and ensure that the values that were deleted (and not updated in the middle), are indeed still deleted. These tests include:

- testDBDeleteMemtable()
- testDBDeleteSST()
- testDBDeletesTwice()
- testDBDeleteReinsert()
- testDBMultipleDeletes()
- testDBMultipleDeletesDeeper()
- testDBPersistentDeletes()
- testDBDeleteScans()

Testing Updates

The test suite also considers the case of updating entries from the database. Not only does it consider the case of updating just within an open database, it also considers the case where we close a database and reopen it and ensure that the values that were updated (and not deleted in the middle), are indeed still updated with their new respective value. These tests include:

- testDBUpdatesMemtable()
- testDBMultipleUpdates()
- testDBMultipleUpdatesDeeper()
- testDBPersistentUpdates()

Experiments

We performed 4 different experiments for our database system: Get performance for different data sizes, Scan performance for different data sizes, Put performance for different data sizes, and a performance comparison of B-Tree vs Binary search for a mixed Get/Scan workload. For the first three experiments, we employ the usage of our implemented B-Tree search (while this configuration setting is varied for the last experiment). Additionally, for all experiments, we test data sizes ranging from 128MB to 1024MB (1GB), while fixing our bufferpool size to 10MB, bloom filter bits-per-entry to 8 bits, and memtable size to 1MB. The results of these experiments, along with a short analysis of each, are detailed below.

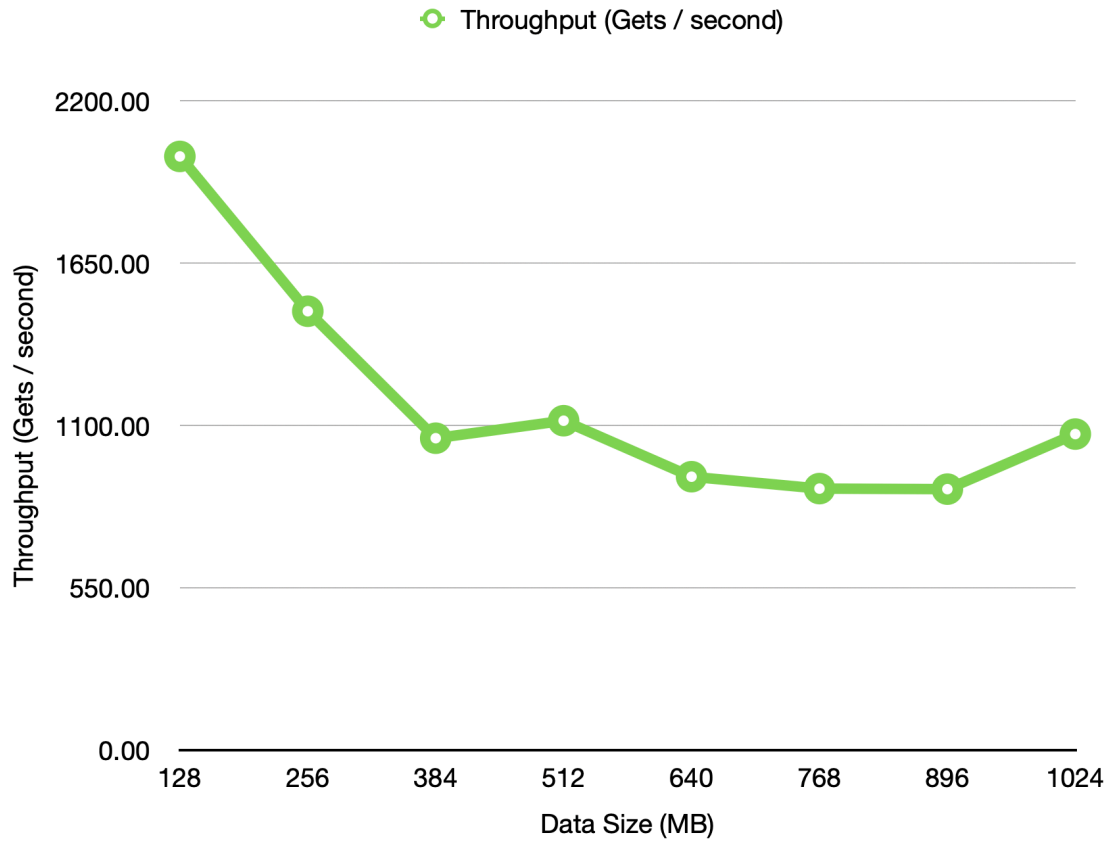


Fig. 1

Figure 1 showcases the throughput (measured in gets/second) for the Get operation in the database. We observe that there is a relatively higher throughput when we deal with smaller data sizes (i.e. a throughput of roughly 2210 gets/second when we have a data size of 128MB). However, as we introduce progressively larger data sizes, we notice that the throughput drops. This is explained via the fact that more data means that it would take longer to get/find the entries that we wish to retrieve. A peculiar phenomenon occurs when we reach the data sizes of 512 MB and 1024 MB (i.e. 1 GB), wherein we note that the throughput of get improves from their respective previous data size values (i.e. 384MB and 896MB). This can be explained by the fact that there is just a single SST that exists during these timeframes, meaning that there are not as many levels that the get algorithm needs to search through in order to retrieve the entry in concern. One would note that the same fact of only having one SST to search through would apply to the 128MB data size case as well. However, the SST that exists in the 128MB case would be situated at a higher level, thereby meaning that there is both a smaller amount of levels to travel and also, a smaller amount of data to search within the level (as due to the scale factoring aspect of LSM trees, the amount of data that resides in these levels are significantly different).

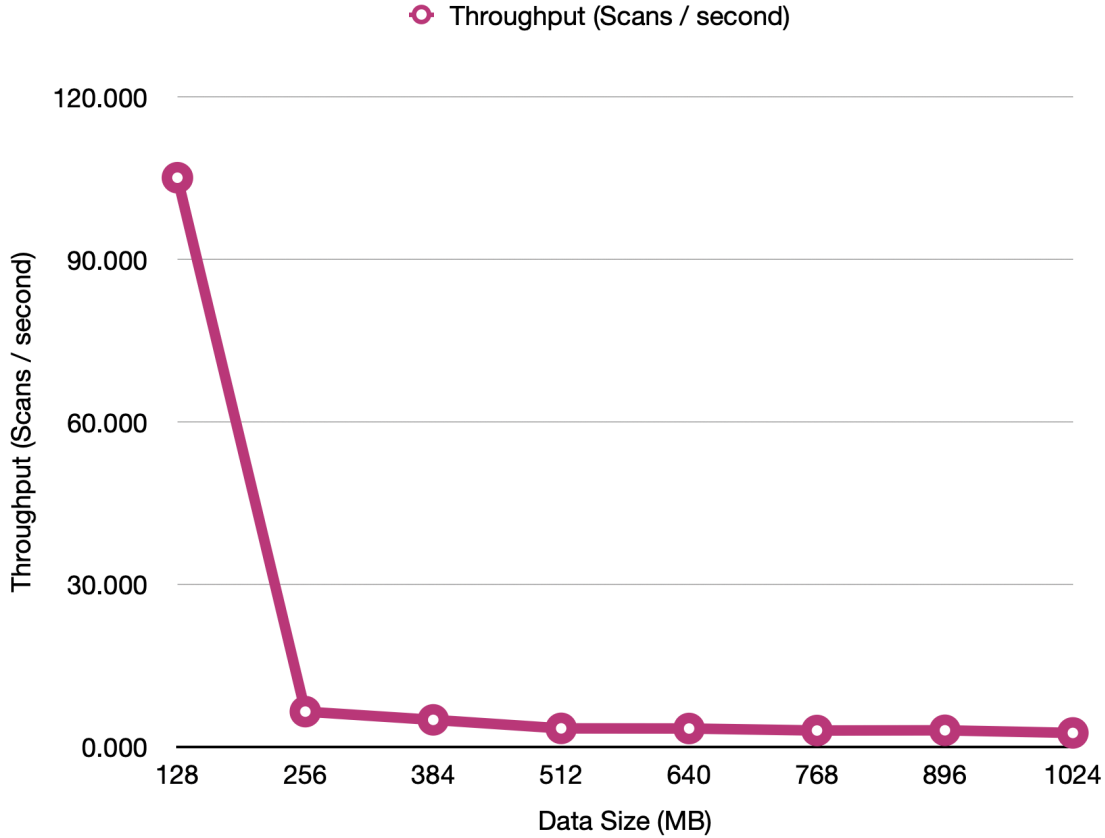


Fig. 2

Figure 2 shows the throughput (measured in scans/second) for the Scan operation in our database. This figure shows that we initially have very high throughput when our database size is 128MB but it has a rapid drop from 256MB onwards. The throughput stays roughly the same as we increase it all the way up to 1GB. We believe at 128MB we have such high throughput because there is only 1 SST to search through at that data size (level 8 SST). However for all other data sizes, we have multiple SSTs that must be read into and searched through which would drop performance of these scans. We also have the obvious observation that more data requires more work to scan through. This phenomenon can be seen as the throughputs slowly decrease as the data size increases from 256MB onwards. Scans in general are also very time consuming compared to gets due to the large range that must be searched which causes throughput to be higher for gets compared to scans.

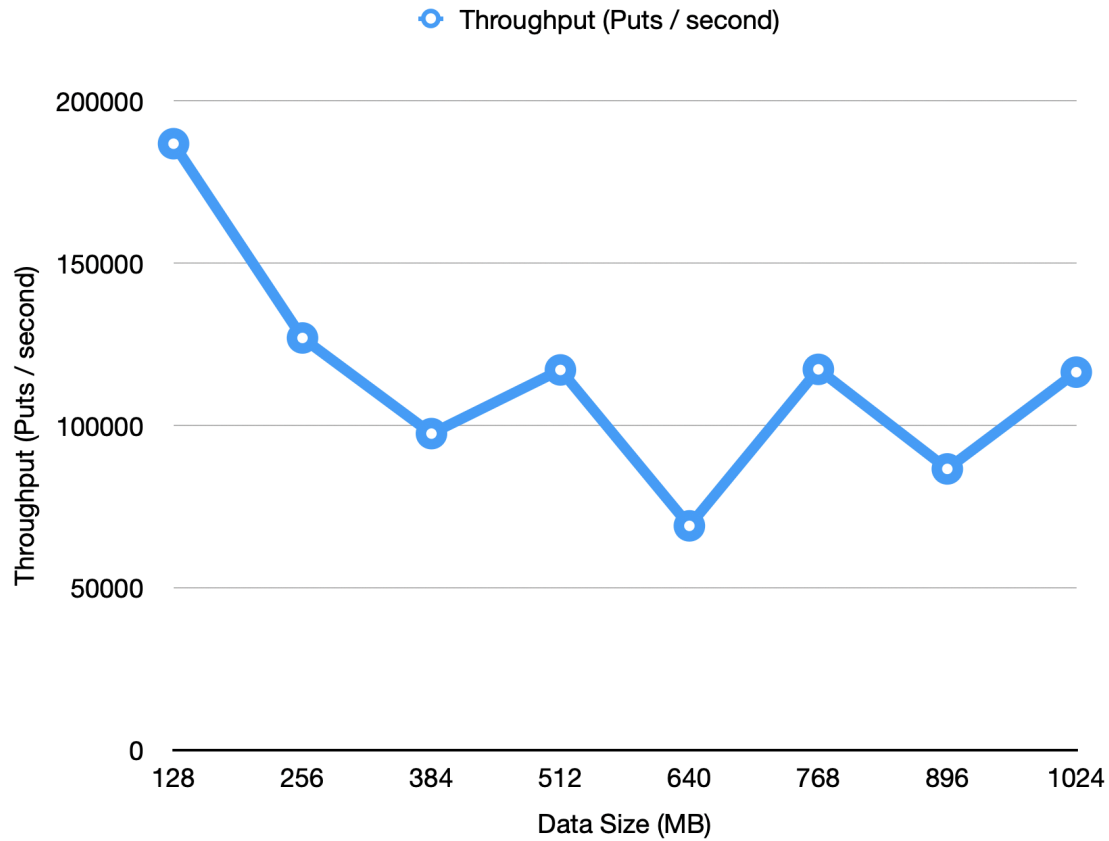


Fig. 3

Figure 3 showcases the throughput (measured in puts/second) for the Put operation in the database. We observe that there is a relatively higher throughput when we deal with smaller data sizes (i.e. a throughput of roughly 19500 puts/second when we have a data size of 128MB). However, as we introduce progressively larger data sizes, we notice that there is a drop in the throughput amount. For instance, we note that the lowest throughput occurs when we reach a data size of 640MB. This can be explained by the fact that there is a relatively larger compaction that is taking place here. The relatively smaller throughput drop in the beginning between the smaller data sizes (i.e. 128MB data size vs. 256MB data size) can be explained by the fact that there are relatively smaller compactions that are taking place here from the SSTs that exist in earlier levels (due to the fact that there is not that much data that has been put into the database yet, resulting in a smaller amount of levels to exist in our LSM tree). However, one would note that we have intermediate peaks that appear at the data size points of 512MB, 768MB and 1024MB. We would expect that there are large chains of SSTs that would be compacted, thereby making the put throughput at these data size points even worse than what they appear to be. However, this phenomenon could be explained by the optimization that we have done where in these large chains of SSTs, there may not be bloom filters and

internal nodes being created for them, thereby offsetting the performance decline that would occur.

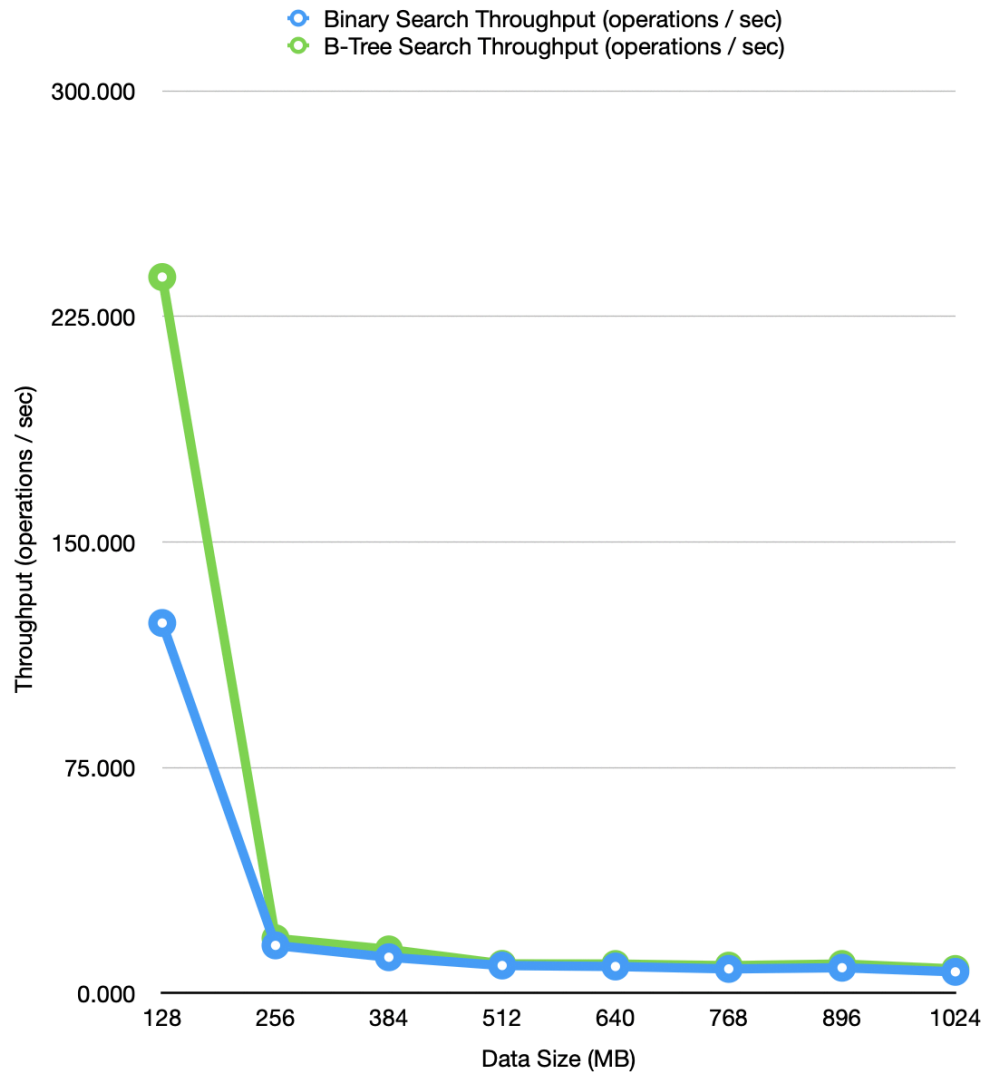


Fig. 4

Figure 4 showcases the difference in throughput (measured as operations/second, where our workload is divided between 50% Gets and 50% Scans) between using B-Tree search and binary tree search for locating pages in the SST. The difference in throughput is very noticeable for smaller data sizes, though they begin to converge as the data size grows larger. However, that said, B-Tree search still always maintains a higher throughput when compared

to binary search. This can be explained by the fact that the runtime of B-Tree search is $O(\log_{256} N)$ (since our branch factor is 256), whereas the runtime of binary search is only $O(\log_2 N)$. For smaller data sizes, where the I/O cost of locating the correct page to Get/Scan from can dominate runtimes, this difference is very valuable and is reflected in the stark difference in throughput. However, the convergence in throughput for larger data sizes can be attributed to other costly parts of the Get/Scan operations, such as needing to read larger SSTs at deeper levels of the LSM tree. We can also surmise that the Scan operations in particular dominate the Get operations in terms of runtime, as can be seen from the similarity of this figure to Fig. 2, which makes sense due to the generally lower throughputs of Scan operations.

Citations

Collet, Y. (n.d.). Cyan4973/xxHash: Extremely fast non-cryptographic hash algorithm. GitHub.

<https://github.com/Cyan4973/xxHash>

Google. (n.d.). Google C++ Style Guide.

<https://google.github.io/styleguide/cppguide.html>